BidOps — Codebase Guide

A plain-language walkthrough of every backend and frontend file in the BidOps repository. Written so you can explain what any part of the code does, in everyday terms, without reading the code itself.

How BidOps Works, in One Paragraph

BidOps is a web app that helps a company decide whether to bid on a government/PSU tender. A user uploads company documents (certificates, employee resumes, past project records) which AI turns into a structured "Capability Library." They also upload a tender document, which AI reads to extract every requirement. The Decision Engine then compares the tender's requirements against the company's capabilities, requirement by requirement, and produces a GO / NO-GO recommendation with evidence for every verdict. The backend (Python/FastAPI) does the data storage and AI orchestration; the frontend (React/TypeScript) is the web interface the user actually clicks through.

Part 1 — Backend (Python / FastAPI)

The backend is the server: it stores data in a database, talks to the AI models, and exposes an API the frontend calls. Organized into folders by responsibility.

This document explains every Python file in `backend/app/` in plain language, so you can walk someone through your own codebase without needing to read code. Files are grouped by folder, in the order data generally flows through the system.

agents/ — The AI Workers

This folder holds the actual "AI" part of BidOps: the code that reads documents, talks to the language model (OpenAI/Gemini/Qwen), and turns messy PDFs/CVs/certificates into structured facts and decisions. Nothing in this folder touches the database directly — it does the thinking, and the services/ folder does the saving. Think of agents/ as the brain and services/ as the hands.

agents/capability_builder.py — Takes one uploaded document (a certificate, employee CV, or project record), extracts its text, asks the AI to pull out structured fields (name, dates, skills, etc.), and computes a confidence score for how trustworthy that extraction is. This is what runs when a user clicks "Build" on a document in the Capability Library.

agents/decision_engine.py — The core reasoning engine that decides, for each tender requirement, whether the company's existing capabilities (certifications, staff, past projects) satisfy it. It calls the AI once per requirement, then applies deterministic business rules on top (risk level, whether human review is needed, an overall Go/No-Go recommendation, and a written summary). This is what powers the "Run Evaluation" step.

agents/document_parser.py — The low-level toolkit for pulling text out of PDFs, Word docs, and images. If a PDF is a real digital document it reads the text directly; if it's a scanned image, it falls back to OCR (optical character recognition) to "read" the picture as text.

agents/json_utils.py — A tiny helper that cleans up the AI's response text (sometimes it wraps its answer in code-formatting) before turning it into structured data.

agents/llm_client.py — The switchboard that connects BidOps to whichever AI provider is configured — OpenAI, Google Gemini, or Qwen (Alibaba). It handles retries when a call fails, translates each provider's own error messages into one common format, and includes a "mock" mode for testing without spending real AI credits.

agents/llm_exceptions.py — Defines a shared set of error types (e.g. "the AI took too long," "bad API key," "rate limit hit") so the rest of the app doesn't need to know which specific AI provider is being used underneath.

agents/mock_extraction.py — A stand-in fake "AI" used only in testing/sandbox mode. It uses simple pattern-matching (not real intelligence) to pull plausible values out of test documents, so the rest of the pipeline can be tested without calling a real, paid AI service.

agents/tender_analyzer.py — Reads an uploaded tender document (which can be dozens of pages) in chunks, asks the AI to extract every requirement (eligibility, technical, certification, deadlines, etc.) along with which page it came from, then removes duplicate entries. This is what runs when a user clicks "Run Analysis" on a tender.

agents/tender_metadata_guess.py — A quick, non-AI helper that scans the first page or two of a newly-selected tender PDF to guess its name, issuing organization, and closing date — just to pre-fill the upload form for convenience. It's allowed to come back empty; it never blocks the actual upload.

agents/prompts/ — The Instructions Given to the AI

This subfolder holds the exact wording sent to the AI model for each task — essentially the "job descriptions" given to the AI so it returns consistent, structured answers instead of free-form prose.

agents/prompts/certification.py — Instructions for extracting fields (name, issuer, dates) from a certification/license document.

agents/prompts/decision_matching.py — Instructions for the AI's core job: deciding whether a specific tender requirement is satisfied by a specific company record, and explaining why.

agents/prompts/employee.py — Instructions for extracting fields (name, role, skills, experience) from an employee CV.

agents/prompts/project.py — Instructions for extracting fields (client, industry, value, status) from a past-project completion document.

agents/prompts/tender_requirement.py — Instructions for pulling a list of requirements out of one chunk of a large tender document, with page numbers attached for traceability.

api/v1/ — The Endpoints the Frontend Talks To

This is the "front door" of the backend — every button click or page load in the BidOps web app ultimately calls one of these endpoints. Each file groups the endpoints for one area of the product (auth, documents, tenders, etc.). These files are intentionally "thin" — they check who's logged in, call the appropriate service, and translate results into HTTP responses; the real logic lives in services/.

api/deps.py (in api/, shared by all v1 routes) — Shared login/permission checks used by nearly every endpoint: confirms a user is logged in with a valid token, and enforces role rules (e.g. only Administrators can add users; only Executives/Administrators can approve a bid decision).

api/v1/router.py — The master switchboard that plugs every other endpoint file into the app under /api/v1/... New endpoint groups get registered here.

api/v1/auth.py — Sign-up, login, and "who am I" endpoints. Sign-up creates a brand-new company and its first admin user together.

api/v1/users.py — Lets an Administrator add teammates to their own company, and lets any logged-in user see the list of colleagues in their company.

api/v1/company.py — Lets a logged-in user view their own company's profile (name, registration number, etc.). Company creation only happens via sign-up, not this endpoint.

api/v1/documents.py — Upload, list, view, download, and delete company documents (certificates, CVs, project records, tender PDFs). Every action is limited to the uploading company's own files.

api/v1/capabilities.py — Triggers the AI extraction of a document into a structured capability record (certification/employee/project), and exposes the "Company Capability Graph" — a full read-out of

everything the company has on file, grouped by type, with freshness/expiry flags. Also handles editing or removing a capability record.

api/v1/tenders.py — Upload a tender document, get a quick AI-free "best guess" of its name/org/deadline for the upload form, view a tender's extracted requirements, and trigger the AI to analyze the tender's requirements in full.

api/v1/evaluation.py — Runs the Decision Engine for a mission (comparing tender requirements against company capabilities) and returns the resulting recommendation, the full compliance matrix (requirement-by-requirement verdicts), and a gap analysis of what's missing.

api/v1/missions.py — A "mission" bundles one tender through its full lifecycle. This file lists missions, fetches one, archives it, and has the single "execute" button that runs the whole pipeline (analyze tender, evaluate, recommend) end to end.

api/v1/approval.py — The human decision-making layer: lets a manager mark individual compliance items as verified/rejected, and record the final Go/No-Go/Conditional-Go decision on a mission, plus view the full approval history/audit trail.

core/ — Foundational Plumbing

Everything the rest of the app depends on but that isn't specific to any one business feature: configuration, the database connection, security, logging, file storage, and rate limiting. If agents/ is the brain and api/ is the front door, core/ is the electrical wiring and foundation of the building.

core/config.py — The single place where all settings live (database address, which AI provider to use, secret keys, upload size limits, etc.), loaded from environment variables. Nothing else in the app is allowed to read environment variables directly — everything goes through here, which also has safety checks that refuse to start in production with insecure default settings.

core/database.py — Sets up the connection to the PostgreSQL database and provides a fresh, safely-closed database session for each incoming web request.

core/logging_config.py — Turns on structured logging for the whole application at startup, so that when something goes wrong in production there's a real trail to investigate, not silence.

core/rate_limit.py — Caps how many times someone can hit expensive or abusable endpoints (like sign-up or AI-processing uploads) per minute/hour from the same IP address, to prevent abuse and runaway AI costs.

core/security.py — Handles password hashing/checking and creation/verification of login tokens (JWTs) — the mechanics behind "log in" and "stay logged in."

core/storage.py — Manages where uploaded files actually live on disk (organized by company), enforces allowed file types and size limits while a file is being uploaded (not after), and handles deleting files later.

models/ — The Database Tables

This folder defines the actual shape of the company's data as stored in the database — every table, column, and relationship. Think of it as the filing cabinet structure: what folders exist, and what goes in each one.

models/__init__.py — A registry file that makes sure every table definition gets loaded together, so the database schema tooling sees the complete picture.

models/mixins.py — Shared building blocks reused across many tables: every table gets a UUID as its unique ID, and every "capability" record (certifications, employees, etc.) shares a common set of tracking fields (created date, last verified date, confidence score, which source document it came from, soft-delete marker).

models/enums.py — The fixed lists of allowed values used throughout the system — user roles, requirement types, match statuses (Met/Not Met/Review Required), risk levels, mission stages, and so on. This is the "controlled vocabulary" of the whole product.

models/company.py — Defines the Company table (the tenant/organization using BidOps) and the User table (people who log in and belong to a company).

models/audit.py — Defines the Audit Log table — a running history of what happened during a mission (which AI agent ran, what it did, whether it succeeded), used for traceability and the approval history screen.

models/document.py — Defines the Document table — every file a company uploads (tender, certificate, CV, project record), including its processing status and where it's stored.

models/capability.py — Defines the five "capability" tables that make up a company's profile: Certifications, Employees, Projects, Equipment, and Financial Records. These are the building blocks the AI checks tender requirements against.

models/mission.py — Defines the Mission table (one full evaluation run for one tender, tracked through stages like Created, Running, Awaiting Approval, Completed) and the Capability Snapshot table (a frozen copy of the company's capabilities at the moment a mission was evaluated, for historical accuracy).

models/recommendation.py — Defines the Recommendation table (the AI's Go/No-Go verdict on a tender, with confidence scores) and the Compliance Matrix table (the requirement-by-requirement scorecard behind that verdict).

models/tender.py — Defines the Tender table (an uploaded bid document), the Requirement table (each individual rule/condition extracted from it), and the Capability Mapping table (which company record was matched against which requirement).

schemas/ — The API's Data Contracts

While models/ defines what's stored in the database, schemas/ defines what's sent back and forth over the web API — the exact shape of request and response data. This is also where incoming data gets validated (e.g. rejecting a malformed sign-up request) before it ever touches the database.

schemas/auth.py — Shapes for login requests, sign-up requests (company + first admin together), and the login response (access token + user info).

schemas/user.py — Shapes for creating a user and for reading back a user's public profile.

schemas/company.py — Shape for reading back a company's profile.

schemas/document.py — Shape for reading back a document's info (deliberately excludes the internal file storage path, for security).

schemas/extraction.py — Defines exactly what fields the AI's extraction results must contain to be considered valid — for certifications, employees, projects, tender requirements, and requirement-matching decisions. If the AI's answer doesn't fit this shape, the extraction is treated as failed rather than silently saving bad data.

schemas/capability.py — Shapes for reading back each of the five capability record types (certifications, employees, projects, equipment, financial records), plus the combined result returned after building a new capability record.

schemas/capability_graph.py — Shapes for the full "Company Capability Graph" view — each capability type combined with freshness info (expired/stale/current), plus a summary count across the whole company.

schemas/revalidation.py — Shapes for editing/updating a capability record and for reporting back which missions were affected and re-evaluated as a result.

schemas/tender.py — Shapes for tenders and their extracted requirements, the upload response, and the quick metadata-guess response used to pre-fill the upload form.

schemas/mission.py — Shape for reading back a mission's status and details (including its linked tender's name), and the optional request body for triggering a mission's execution (e.g. choosing which AI engine to use).

schemas/decision.py — Shapes for the Decision Engine's output: the compliance matrix (with full evidence trail — which company document backs up each verdict), the recommendation itself, and the gap analysis (what's missing or needs review).

schemas/approval.py — Shapes for a human's verification of a specific compliance item, the final Go/No-Go decision, and the full approval history (including a required written reason when rejecting a bid).

services/ — The Business Logic

This is the busiest folder — it's where the actual rules of the business live: how a sign-up works, how a document gets saved and linked to a company, how a mission moves through its stages, and how a capability change automatically re-triggers evaluations elsewhere. api/v1/ files call into these; these files call into models/ (to save/read data) and agents/ (to call the AI). This separation keeps the "what does the business do" logic separate from "how does the API answer a web request."

services/exceptions.py — Defines the shared set of business error types (Not Found, Conflict, Authentication Failed, File Too Large, etc.) that every service raises, which the API layer then translates into proper HTTP error responses.

services/auth_service.py — Handles sign-up (creating a new company and its first admin user together, as one atomic action) and login (checking credentials and issuing an access token).

services/user_service.py — Creates new users within a company and looks users up by ID or email.

services/company_service.py — Looks up a company's details by ID.

services/freshness.py — Calculates, on the fly, whether a capability record (like a certificate) is expired or "stale" (too old to fully trust) based on its expiry date and how long ago it was last verified. Nothing here is saved to the database — it's recalculated every time it's needed.

services/document_service.py — Handles uploading a document (validating file type, saving it to disk, recording it in the database), fetching/listing a company's documents, and safely deleting a document (blocking deletion if it's still actively referenced by a live tender or capability record).

services/capability_service.py — The persistence layer for the AI extraction pipeline: kicks off document extraction, saves the resulting capability record, prevents building duplicate capabilities from the same document, and provides the read/list functions that power the Capability Graph. Also handles editing and soft-deleting capability records.

services/tender_service.py — Handles uploading a tender (creating both the Tender record and its parent Mission together), the quick non-AI metadata guess for the upload form, fetching a tender and its requirements, and running the full AI requirement-extraction analysis on a tender.

services/mission_service.py — The workflow orchestrator: lists/fetches/archives missions, and drives the single "Execute Mission" action that runs tender analysis and decision evaluation in the correct order, skipping steps that are already done, and logging each step to the audit trail.

services/decision_service.py — The persistence layer for the Decision Engine: runs the AI matching for every requirement in a tender (in parallel, with a safety cap on how many run at once), computes the overall recommendation and confidence scores, and saves everything (snapshot, recommendation, compliance matrix rows) to the database in the correct order. Also resolves the "evidence trail" — linking a recommendation back to the specific company document that backs it up.

services/approval_service.py — The human decision-making backend: lets a manager verify or reject individual flagged compliance items, records the final Go/No-Go/Review decision on a mission (blocking finalization if high-risk items are still unverified), and assembles the full approval history for a mission.

services/revalidation_service.py — Keeps recommendations up to date automatically: when a capability record is edited, removed, or becomes stale/expired, this module traces which missions relied on it and re-runs their evaluation, without ever overwriting a decision a human has already made on a completed mission.

Top-Level

main.py — The application's actual starting point. This is what boots up the whole backend: it turns on logging, creates the web server, configures which frontend domains are allowed to call it (CORS), turns on rate limiting, and plugs in all the API endpoints. Also exposes a simple /health check used to confirm the server is alive.

Part 2 — Frontend (React / TypeScript)

The frontend is everything the user actually sees and clicks in the browser. It calls the backend's API for data and renders every screen of the product.

This document walks through every file in frontend/src, explaining in everyday terms what it does and how it fits into the app. BidOps is a web app that helps a procurement/bidding team decide whether to pursue a tender (a contract opportunity): it reads tender documents, compares them against the company's own certifications/projects/staff, and produces a GO/NO-GO recommendation with evidence.

Top-level files

These two files are the "ignition switch" for the whole app — they don't show anything themselves, they just wire everything else together.

App.tsx — This is the app's traffic map. It decides which screen (page) shows up for which web address, e.g. "/documents" shows the Documents page, "/missions" shows the Tender Workspace. It also enforces the login wall: if you're not signed in, you get redirected to the Login page (or the public marketing homepage) instead of the real app.

main.tsx — The very first file that runs when the app loads in a browser. It finds the empty root element in the HTML page and tells React to start rendering the whole app (App.tsx) inside it. Nothing else lives here.

vite-env.d.ts — A small technical helper file that tells the build tool (Vite) what environment variables (like the backend server address) are expected to exist. Not something a founder would ever need to open.

api/ — talking to the backend server

This folder is the app's "phone line" to the backend server. Every time the app needs data (list of documents, run an analysis, log in, etc.), the request goes through these three files. Nothing here draws anything on screen — it's pure plumbing.

api/client.ts — Sets up the connection to the backend server. It automatically attaches the user's login token to every request (so the server knows who's asking), and if the server ever says "you're not logged in" (401 error), it automatically clears the session and bounces the user back to the Login page. Also has a helper that turns technical server error messages into readable sentences the app can show the user.

api/endpoints.ts — A menu of every action the app can ask the backend to do: log in, register a company, upload a document, build a capability record, upload a tender, run the AI analysis, run the decision engine, list missions, delete things, etc. Every page in the app calls functions from this file rather than talking to the server directly.

api/types.ts — A dictionary of the exact "shapes" of data the backend sends back (a user, a document, a tender, an evaluation result, etc.). This keeps the frontend and backend speaking the same language — if the backend changes what it sends, this file needs to be updated to match.

context/ — app-wide shared state

React "context" is a way to share information across the whole app without passing it manually from screen to screen. These three files hold three different pieces of app-wide state.

context/AuthContext.tsx — Remembers who is currently logged in (stored in the browser's local storage so it survives a page refresh) and provides the login/logout actions. Every page that needs to know "who is the current user" or "are they logged in" reads from here.

context/ThemeContext.tsx — Remembers whether the app is in light mode or dark mode, and lets any part of the app flip the switch. Defaults to light mode.

context/ToastContext.tsx — Powers the small pop-up notification banners that appear in the corner of the screen (e.g. "Document uploaded successfully" or an error message). Any page can trigger one of these "toasts" without having to build its own notification UI.

lib/ — small reusable helper functions

Utility functions that do one specific calculation or transformation, reused across multiple pages so the logic only needs to be written once.

lib/cn.ts — A tiny helper for combining CSS class names cleanly. Purely a code-styling convenience, invisible to end users.

lib/complianceMerge.ts — Combines two pieces of server data (the compliance checklist and the gap analysis) into one merged list so that every row in the Decision Engine results page has full, readable requirement text instead of raw IDs. Used on both the Decision Engine page and the Reports page so they show identical information.

lib/pdfReport.ts — Generates the downloadable PDF report a user gets when they click "Download PDF Report" on the Reports page. Builds a branded, multi-section PDF (executive summary, confidence scores, risk gaps, full compliance matrix) entirely in the browser, no separate server call needed.

lib/recommendationLabels.ts — Translates the backend's internal recommendation codes ("go", "no_go", "conditional_go", "review") into friendlier business language ("Proceed", "Do Not Proceed", "Proceed with Conditions", "Review Required") wherever a recommendation is shown on screen.

lib/tenderName.ts — Decides what name to display for a tender: the name the user typed when uploading it, or (if they left it blank) the original file name. Ensures tenders never show up with a confusing internal code as their name.

components/ (top-level) — shared app frame

components/Layout.tsx — The persistent frame around every logged-in page: the left sidebar with navigation links (Dashboard, Documents, Capabilities, Upload Tender, Tender Workspace, Reports), the top header bar (page title, dark-mode toggle, live clock, and the account menu with Log Out), and the main content area where each individual page is displayed. Every authenticated page in the app is shown inside this frame.

components/kit/ — the design system (reusable building blocks)

This folder is the app's "Lego set" — small, generic, reusable visual pieces (buttons, cards, badges, form fields) that every page is built out of. Keeping these in one place means the whole app looks

and behaves consistently, and a style change only has to be made once.

kit/index.ts — Just a list that re-exports everything in this folder from one place, so other files can import multiple kit pieces in a single line.

kit/Button.tsx — The standard clickable button used everywhere (primary, outline, danger styles, small/medium/large sizes, with a built-in loading spinner state).

kit/Card.tsx — The white rounded-corner box used to group content on every page, plus a header/body layout and a "StatCard" variant used for the small number tiles (like "Total Documents: 12") seen on Dashboard, Documents, and Capabilities.

kit/Badge.tsx — The small colored pill/label used to show status everywhere (e.g. "GO" in green, "NO-GO" in red, "Pending" in gray). Has one central rulebook mapping every status word in the app to a consistent color, so "completed" or "met" always looks the same shade of green no matter which page it's on.

kit/ConfidenceBar.tsx — Draws the confidence percentage bars and circular "confidence ring" gauges seen on the Decision Engine and Reports pages (e.g. an 82% circle showing how confident the AI is in its recommendation).

kit/Dropzone.tsx — The drag-and-drop file upload box used on the Documents and Upload Tender pages. Shows an upload icon and instructions when empty, and shows the selected file's name/size with a remove button once a file is chosen.

kit/Form.tsx — Standard text input and dropdown ("select") field styles used across every form in the app (login, registration, document upload, tender upload, etc.), so all form fields look and behave the same.

kit/Menu.tsx — The dropdown menu component used for things like the account menu (top-right) and the "more actions" menu on table rows. Handles opening, closing when you click elsewhere, and positioning itself correctly on screen.

kit/SearchInput.tsx — The search box (with a magnifying glass icon and clear button) used on pages like Documents and the Decision Engine results to filter long lists. Also includes the small pill-shaped "filter chip" buttons used for status filters.

kit/Skeleton.tsx — The gray shimmering placeholder blocks shown while a page is still loading data from the server, so the screen doesn't look empty/broken during the brief wait.

kit/EmptyState.tsx — The friendly "nothing here yet" message shown when a list has no items (e.g. "No documents yet, upload your first document above"), used consistently across every page instead of each page inventing its own blank-state message.

kit/AIProcessing.tsx — The "AI is thinking" screen shown while a real AI analysis call is running (building capabilities, analyzing a tender, running the decision engine). Cycles through honest status messages like "Reading document…" rather than a fake progress bar, since the app has no way to know real progress.

kit/Clock.tsx — Powers the live clock shown in the header, and a "Good morning / Good afternoon" style greeting used on the Dashboard, both of which update automatically as time passes.

kit/Logo.tsx — The BidOps logo mark (the little gradient icon) and the full logo-plus-wordmark combo, used in the sidebar, login page, and marketing site header.

kit/Switch.tsx — The on/off toggle switch component, currently used for the light/dark mode control.

kit/StatusDonut.tsx — Draws the colored donut/pie chart used to visually break down compliance results by status (met/conditional/review/not-met) with a percentage in the center.

components/landing/ — the public marketing website

Everything a visitor sees before they log in — the marketing homepage that explains what BidOps is and tries to convert a visitor into a demo request or sign-up. None of this touches real customer data; it's all static marketing content.

landingData.ts — Not a visual component — this is where all the marketing website's text content lives (feature descriptions, industry list, "how it works" steps, trust statements, etc.), kept separate from the visual layout files so the copy is easy to review and edit without digging through page code.

LandingNavbar.tsx — The sticky navigation bar at the top of the marketing site: logo, dropdown menus (Solutions/Features/How It Works), Pricing and Contact links, and Login/Book a Demo buttons. Also handles the mobile hamburger menu version.

SolutionsMegaPanel.tsx — The expandable dropdown panel (shown when you click "Solutions" in the navbar) listing the industries BidOps serves (construction, manufacturing, government, healthcare, etc.) with expandable "Learn more" details for each.

FeaturesMegaPanel.tsx — The expandable dropdown panel for "Features", a filterable grid of BidOps's core capabilities (document analysis, compliance matrix, GO/NO-GO scoring, etc.), grouped into categories you can click through.

HowItWorksMegaPanel.tsx — The expandable dropdown/section showing the 7-step journey from "upload documents" to "get a decision," including small illustrative mock screenshots of each step so a visitor can see roughly what the real product looks like.

DashboardPreview.tsx — A static, fake-but-realistic mockup of the logged-in dashboard, used purely as an illustration on the marketing homepage and login screen to show visitors "this is a real product" without exposing any real data.

ContactSection.tsx — The "Get in Touch" section near the bottom of the marketing homepage: real contact email, a "Request a Demo" link, and a contact form that opens the visitor's email client pre-filled with their message (there's no backend contact-form endpoint, it's a mailto link).

pages/ — the actual screens of the product

This is the heart of the application — one file per screen a logged-in user actually works in day to day, plus the Login and public Landing pages.

pages/Landing.tsx — The public marketing homepage shown at the root web address to anyone who isn't logged in. Combines the navbar, a hero section pitching the product, a trust/credibility strip, the contact section, and a footer. This is what a prospective customer sees first.

pages/Login.tsx — The sign-in and company-registration screen. Has two modes toggled by a link: "Login" (email + password) and "Register" (create a new company account with an admin user).

Includes a marketing-style pitch panel and dashboard preview alongside the actual form.

pages/Dashboard.tsx — The home screen after logging in, this is what a user clicking "Dashboard" in the sidebar sees. Shows five summary number-cards (Tenders, Evaluations, Capability Library size, Reports, Critical Gaps), a table of recent tenders with quick actions, a box showing whether any analysis is currently running, and a recent-activity feed built from real upload/completion events (no invented data).

pages/Documents.tsx — The "Company Documents" page, reached from the Documents sidebar link. Lets a user upload company documents (certifications, resumes, project records, etc.) that will later be turned into capability records, and shows a searchable table of everything already uploaded with status and a delete option.

pages/Capabilities.tsx — The "Capability Library" page. This is where uploaded documents get converted by AI into structured company capability records: certifications, employees, projects, equipment, and financial records. Shows a list of documents ready to "Build," and, once built, organized sections for each capability type with freshness status (current/stale/expired) and an admin-only delete option.

pages/TenderUpload.tsx — The "Upload Tender" screen, reached by clicking "Upload Tender" in the sidebar. Lets a user drag-and-drop a tender PDF and optionally fill in its name, issuing organization, and closing date, the app tries to auto-fill those fields itself by skimming the PDF. Submitting kicks off a new "mission" (the app's internal name for one tender's evaluation journey) and sends the user to Tender Workspace.

pages/TenderDetail.tsx — A detail screen for one specific tender document, showing the tender's basic info and, once analyzed, every extracted requirement (with type, mandatory flag, and confidence score), filterable by requirement type. Has a "Run Tender Analyzer" button to kick off AI extraction and a "Run Decision Engine" call-to-action once requirements exist.

pages/Missions.tsx — The "Tender Workspace" page (sidebar label: "Tender Workspace"), the main hub for tracking every tender's progress. Shows every mission as a card with a 4-step visual progress tracker (Uploaded, AI Processing, Awaiting Approval, Completed), search/filter/sort controls, a "Run Full Analysis" button (which chooses an AI engine and kicks off the whole extraction + decision process in one click), and a delete (archive) option per tender.

pages/Evaluation.tsx — The "Decision Engine Result" screen, the most detailed analysis page in the app, opened by clicking into a mission. Shows the big GO/NO-GO verdict with a confidence ring, a written executive summary, four confidence sub-scores, a risk summary, a "What's Blocking This Bid" callout for any unmet mandatory requirements, and the full requirement-by-requirement compliance matrix (searchable, filterable, grouped by status) with an expandable "evidence trail" showing exactly which document/page/company record backs up each verdict.

pages/Reports.tsx — The "Reports" page, for browsing and exporting finished evaluations. Shows a list of every tender that has a completed recommendation on the left, and on the right a preview of the selected report (verdict, confidence scores, a compact stats row of Reviewed/Matched/Needs Review/Missing, and a compliance summary) with a "Download PDF Report" button that produces a polished, brandable PDF for offline sharing.